

# RELATÓRIO TÉCNICO

## Justificativas das decisões arquiteturais

Autores: Alexandre Della Mônica Moreira e Daniela A. Fontana

### 1. Introdução

O EventMaster é um sistema de venda de ingressos online desenvolvido como projeto prático para aplicar conceitos de arquitetura de software em um cenário realista. A escolha desse domínio não foi aleatória: venda de ingressos envolve picos de acesso imprevisíveis, integrações com *gateways* externos e fluxos transacionais que não toleram inconsistência. Por isso, o projeto acabou servindo como um bom cenário para avaliar decisões arquiteturais que realmente impactariam um ambiente real.

Este relatório descreve as principais decisões tomadas ao longo do projeto e o raciocínio por trás de cada uma. Não se trata apenas de listar o que foi usado, mas de explicar por que cada escolha faz sentido para esse contexto específico e onde ainda existem limitações ou pontos que evoluíram numa versão mais completa do sistema.

As seções seguintes cobrem desde a divisão em contextos delimitados e a escolha por microsserviços, até decisões mais específicas como padrões de resiliência, modelo de dados e segurança.

### 2. Design e Estrutura

Dividir o EventMaster em contextos delimitados como, por exemplo, em microsserviços, é muito mais que uma simples organização técnica, pois essa separação reflete a forma como cada área do negócio funciona, evolui e opera no dia a dia. Em um sistema de venda de ingressos, catálogo, pedido e pagamento possuem objetivos diferentes, comportamentos distintos sob alta demanda e necessidades próprias de manutenção. Centralizar tudo em um único monolito aumenta o acoplamento entre as áreas que deveriam evoluir separadamente, além de criar dependências desnecessárias entre processos que não possuem relação direta entre si.

O primeiro contexto definido foi o Catálogo de Eventos, responsável pelo cadastro, consulta e controle de disponibilidade dos eventos, lotes e ingressos. A separação desse domínio acontece porque o catálogo concentra grande volume de

leitura, principalmente em períodos de alta procura. Em lançamentos de grandes eventos populares, por exemplo, milhares de usuários podem acessar simultaneamente as páginas apenas por curiosidade e esse volume não deveria afetar os fluxos dos usuários que realizarão realmente uma compra e o pagamento. Manter esse serviço separado facilita também para escalar somente as consultas quando necessário, sem precisar aumentar toda a estrutura do sistema.

O segundo contexto é o de Venda e Pedido, responsável pela criação de pedidos, reserva de ingressos e acompanhamento do estado da compra. Esse domínio concentra regras críticas do negócio, já que qualquer inconsistência pode causar reservas duplicadas, perda de ingressos ou falhas no processamento do pedido. Separar essa responsabilidade reduz o risco de efeitos colaterais vindos de outros contextos e deixa mais claro quais regras pertencem ao processo de venda. Além disso, evita que mudanças em catálogo ou pagamento afetem diretamente o fluxo transacional mais sensível do sistema.

O terceiro contexto é o de Pagamento, Essa separação se justifica porque o processamento financeiro depende de serviços externos, como *gateways* de pagamento, que comumente causam instabilidades fora do controle da aplicação. *Timeouts*, indisponibilidade temporária e respostas inconsistentes são situações que acontecem com frequência nesses tipos de integrações. Se isolarmos esse contexto, o sistema consegue diminuir o impacto dessas falhas e aplicar estratégias específicas, como *retry*, controle de transações e mecanismos de tolerância a falhas, sem comprometer os outros domínios. Dessa forma, um problema no *gateway* não derrubaria a aplicação toda.

A API Gateway funcionaria como porta de entrada única para qualquer requisição que vem do usuário. Centralizando a autenticação e roteamento nesse ponto evitaria que cada microserviço precisasse lidar com isso por conta própria, além de impedir que a estrutura interna fique exposta diretamente. Em um sistema com picos de acesso, como aconteceria durante a abertura de vendas de um evento grande, ter esse controle centralizado faria diferença, tanto na organização do tráfego quanto na capacidade de proteger a aplicação contra ataques.

O Circuit Breaker foi pensado principalmente para a comunicação entre pedido e pagamento. O problema que geralmente acontece é que se o serviço de pagamento começa a falhar repetidamente, sem nenhum mecanismo de proteção, isso começa a travar o serviço de pedidos também, e o efeito se propaga. O circuito interrompe temporariamente as tentativas quando detecta instabilidade, dando tempo pro serviço se

recuperar sem derrubar o restante da operação. Quando o pagamento depende de *gateways* externos que podem sair do ar, essa proteção é muito importante.

Adotamos o padrão SAGA para garantir a consistência entre a reserva de ingressos e o processamento do pagamento. Como cada serviço tem seu próprio banco de dados, uma transação distribuída não se encaixaria bem aqui. Isso ficaria contrário à ideia de independência dos microsserviços. A SAGA trata o processo em etapas: reserva o ingresso, tenta processar o pagamento e, se algo falhar, seja por recusa, *timeout* ou erro externo, ele executa as compensações necessárias para desfazer o que já foi feito e liberar o estoque.

A implementação por coreografia é a que mais faz sentido nesse cenário porque cada serviço reage a eventos e dispara o próximo passo sem depender de um orquestrador central. Isso mantém o fluxo distribuído e reduz o acoplamento entre os serviços. O ponto de atenção é que, sem um controlador central, rastrear o estado do processo exige cuidado na modelagem dos eventos, do contrário, fica difícil saber em que etapa uma compra travou ou gerou erros.

### 3. Processamento e Performance

A divisão entre Stream e Batch no projeto foi feita avaliando em quais pontos deveríamos ter processamento e respostas em tempo real e em quais pontos seria um processamento de grandes quantidades de informação, usando principalmente lotes.

Os fluxos orientados a Stream cobrem as operações diretamente ligadas às ações do usuário e ao estado da compra. No serviço de pagamento, eventos como aprovação, rejeição ou geração de boleto são publicados em tópicos Kafka para que os demais serviços reajam sem depender de chamadas síncronas entre si. Isso elimina a ligação direta entre os domínios e evita que um serviço fique bloqueado esperando resposta de outro.

Esse modelo se conecta diretamente ao padrão SAGA que adotamos no projeto. Quando um pedido é criado, o fluxo ocorre em etapas: reserva do ingresso, processamento do pagamento e, dependendo do resultado, confirmação ou cancelamento da operação. Se o pagamento falhar, eventos de compensação liberam o estoque sem precisar de uma transação distribuída entre múltiplos bancos.

O reprocessamento de pagamentos também segue essa mesma lógica. Quando um pagamento fica em estado aguardando o pagamento ou sofre alguma inconsistência, o sistema publica eventos de reprocessamento sem interromper o restante da aplicação.

Isso isola o problema e evita que a indisponibilidade de um *gateway* externo interfira no processamento dos outros domínios.

Para Batch, deixamos as operações onde o imediatismo não é necessário e onde precisamos trabalhar com volume de dados. A expiração de boletos é o exemplo mais direto: não há motivo para verificar cada boleto no exato momento do vencimento. Um *job* agendado resolve isso com muito menos custo de processamento e sem impactar os serviços transacionais.

O mesmo vale para os relatórios financeiros, totais de venda, taxas, estornos e inadimplência, entre outros. Eles dependem do processamento de grandes volumes de dados e rodar isso continuamente seria desperdício. Um processamento diário bem estruturado entrega a mesma informação com muito menos pressão sobre a infraestrutura.

## 4. Segurança

A autenticação no EventMaster foi desenhada considerando que o projeto, no estágio atual, implementa apenas o *payment service*. Isso significa que boa parte dos mecanismos de segurança ainda não aparece no código, mas as decisões arquiteturais já foram tomadas e refletem como o sistema funcionaria num cenário completo.

A proposta usa OAuth 2.0 com JWT. O fluxo seria centralizado num serviço de autenticação dedicado, responsável por validar credenciais e emitir *tokens*. A API Gateway receberia todas as chamadas externas, validaria o *token* antes de encaminhar qualquer requisição e o *payment service* receberia apenas chamadas já autenticadas, sem precisar implementar login ou controle de sessão por conta própria. Isso evita duplicar responsabilidade em cada serviço.

O *webhook* de boleto é um caso diferente. Como as chamadas vêm de *gateways* bancários externos e não de usuários autenticados, JWT não se aplica aqui. A validação seria feita por outros mecanismos por exemplo, assinatura HMAC, *tokens* secretos compartilhados ou restrição por IP, dependendo do que o *gateway* utiliza. Esse *endpoint* ainda não tem validação de origem implementada, o que é uma limitação real do estágio atual.

Em relação às práticas de segurança, algumas já estão presentes no código. A validação de dados acontece em múltiplas camadas: nos DTOs com anotações como `@Valid`, `@NotNull` e `@DecimalMin`, e também dentro do próprio domínio. O sistema também controla idempotência no fluxo de pagamento, antes de criar um novo

pagamento, verifica se já existe um com o mesmo pedido, o que previne duplicidade em cenários de *retry* ou falha de rede.

O padrão Outbox também contribui para a consistência. Eventos de pagamento são persistidos junto da transação principal antes de serem publicados no Kafka, o que elimina a janela de inconsistência entre banco e mensageria. A separação entre domínio e integrações externas via *ports and adapters* também ajuda a isolar responsabilidades e trocar o *gateway* de pagamento, por exemplo, não exigiria mexer nas regras de negócio.

Um ponto que ainda precisa evoluir é o tratamento de dados de cartão. Hoje a entidade armazena essas informações no banco, mas os DTOs de resposta já não expõem número do cartão nem CVV. Em produção, o caminho seria tokenização completa, eliminando o armazenamento direto e alinhando com PCI DSS. Outros pontos em aberto: *endpoints* sem autenticação real, Kafka e banco sem criptografia no ambiente de desenvolvimento. São limitações conhecidas do estágio atual, não do modelo arquitetural.

O conceito de Zero Trust também foi considerado no desenho. A ideia do conceito é justamente não confiar automaticamente em um serviço só porque ele está dentro da rede interna. Cada comunicação deveria ter autenticação própria e validação explícita de permissões. Na versão atual isso ainda não está implementado, mas a divisão em microsserviços e a centralização da autenticação no *gateway* já apontam nessa direção.

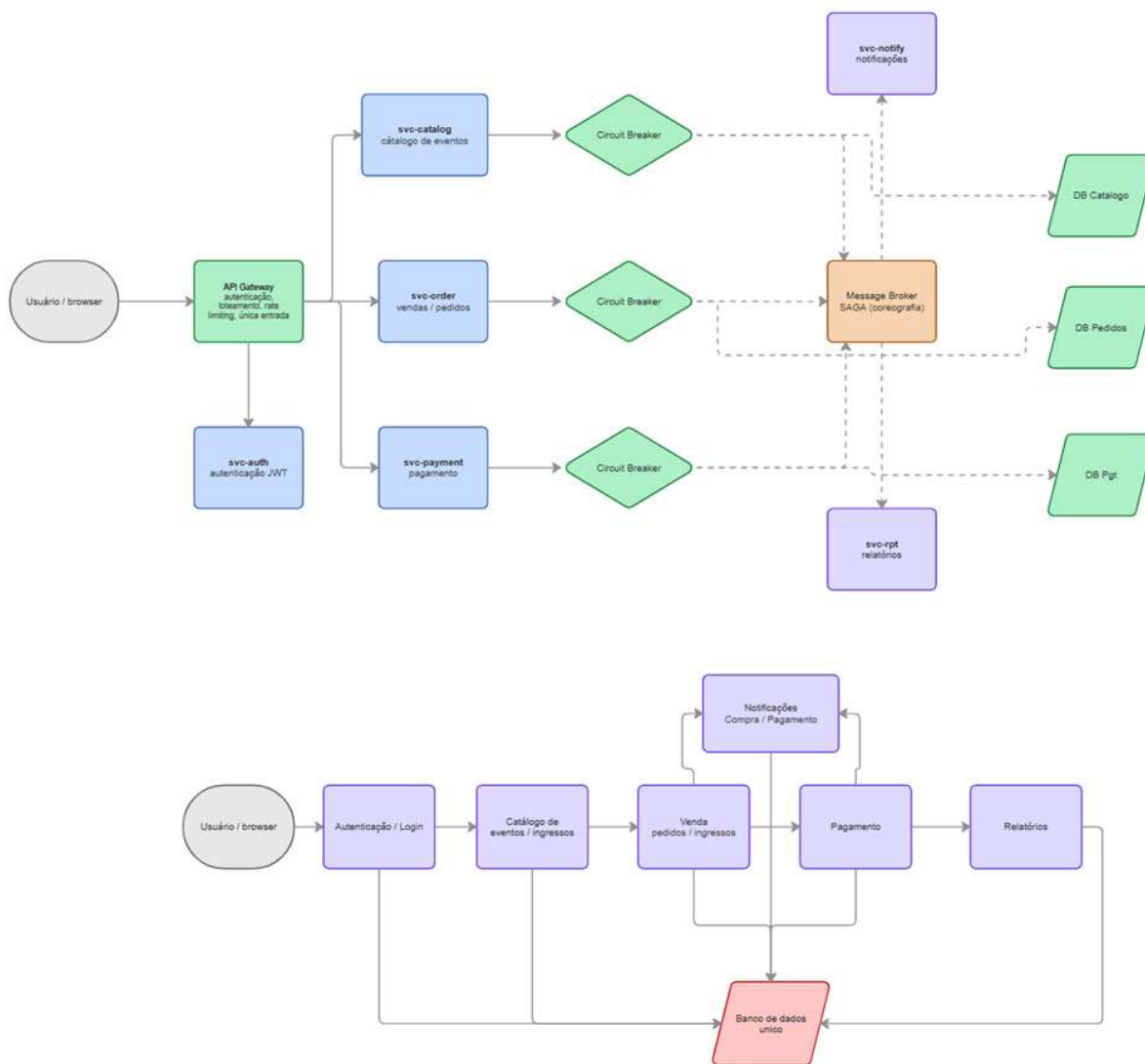
## 5. Conclusão

As decisões tomadas no EventMaster partiram de um problema concreto: sistemas de venda de ingressos falham exatamente quando mais importa, no momento de maior acesso. Esse contexto guiou as escolhas ao longo do projeto, desde a separação em microsserviços até decisões mais específicas como SAGA, Circuit Breaker e a divisão entre Stream e Batch. Cada uma dessas escolhas existe porque havia um problema real por trás dela, não apenas uma boa prática a seguir.

O projeto implementa atualmente apenas o *payment service*, mas a estrutura já reflete como o sistema completo funcionaria. As bases estão definidas de forma que novos serviços possam ser adicionados sem comprometer o que já existe. Num projeto real, é exatamente esse tipo de decisão antecipada que evita reescritas custosas mais à frente.

# ANEXOS

Figura1 - Transição de Monolito para Microserviços



Fonte: Elaborado pelo autor (2026).